
Universidad del Valle
Facultad de Ingeniería
Escuela de Ingeniería de Sistemas y Computación
COMPUTACIÓN DISTRIBUIDA y FRAMEWORKS para DESARROLLO de APLICACIONES INTELIGENTES
Escuela de Ingeniería de Sistemas y Computación

Despliegue de Servicios de Red



Tabla de Contenido

Resumen 4
Procesos de cómputo como servicios de red 5
Escenario 6
Despliegue de procesos en ambientes virtuales a nivel del sistema operativo 8
Contexto 8
Práctica 8
Creación de servicios a partir de procesos ejecutados en ambientes virtuales 12
Contexto 12
Práctica 12
Publicación en repositorios públicos de ambientes virtuales 16
Creación y preparación de entornos virtuales en ambientes de nube pública (máquinas virtuales) 18
Acondicionamiento de la máquina virtual 26
Finalmente y no menos importante, de hecho MUY IMPORTANTE 28
Referencias 31



Resumen

Este taller tiene como objetivo comprender y desplegar un servicio de red disponible en un ambiente de nube pública. Se llevará a cabo la comprensión e interiorización de conceptos relacionados con:

- Procesos de cómputo como servicios de red
- Despliegue de procesos en ambientes virtuales a nivel del sistema operativo
- Creación de servicios a partir de procesos ejecutados en ambientes virtuales
- Exposición de servicios a través de la red y ejecutados en ambientes virtuales
- Publicación en repositorios abiertos de ambientes virtuales a nivel del sistema operativo
- Creación y preparación de entornos virtuales en ambientes de nube pública
- Despliegue de ambientes virtuales localizados en repositorios en ambientes de nube pública

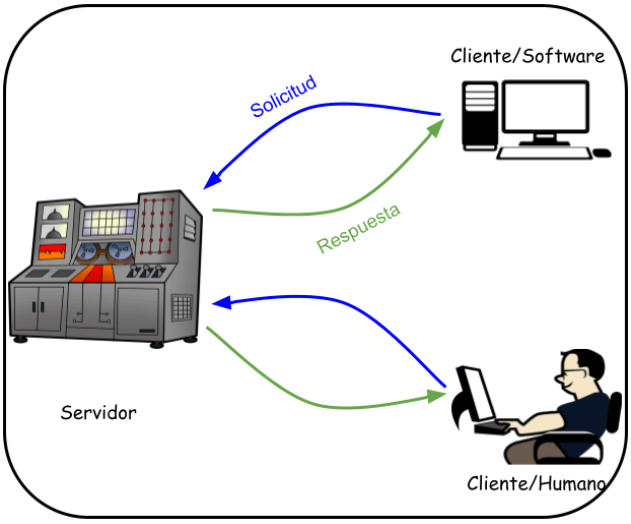
Procesos de cómputo como servicios de red

Los servicios servicios de red son procesos que se ejecutan en un computador con la capacidad de atender peticiones a través de la red.

Los servicios de red son la base sobre la cual opera Internet pues son estos servicios los que nos permiten llevar a cabo actividades relacionadas con la comunicación, la productividad, la formación y el esparcimiento.

Los servicios de red son aplicaciones que llevan a cabo algún tipo de tarea y la solicitud de la ejecución de este tipo de tarea la puede solicitar un usuario u otro proceso de red que así lo indique.

La forma de interacción entre un cliente (humano o programa) y un servidor se puede observar en esta gráfica.



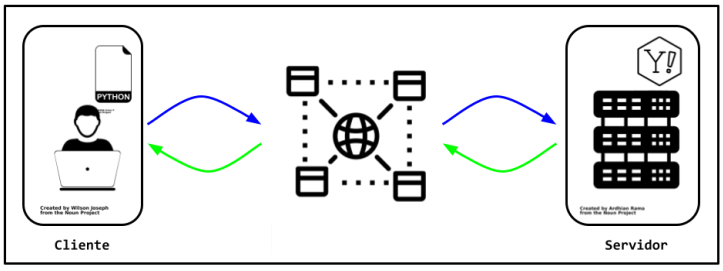
Escenario

Suponga que usted está interesado en las inversiones y quiere tener la posibilidad de consultar a través de un **comando** como está su *stock market* de preferencia. El algoritmo de este problema es muy sencillo:

```
- traer datos de su stock market preferido de un tiempo_inicial a un tiempo_final
- sacar el valor promedio de dicho stock market
```

Veamos cómo sería esto en el lenguaje de programación Python[1]

```python
# Importing the yfinance package
import yfinance as yf
# Set the start and end date
start_date = '2020-01-01'
end_date = '2022-01-01'
# Set the ticker
ticker = 'GOOGL'
# Get the data
data = yf.download(ticker, start_date, end_date)
# Print the last 5 rows
print(data.tail())
```



En la gráfica se muestra una representación de la interacción entre el script en Python y su acceso a la infraestructura de Yahoo donde se ejecuta el servicio Finance y que lleva el registro de la actividad bursátil a nivel mundial. En la interacción cliente-servidor, el cliente ejecuta el script y este se encarga de contactar al servidor de Yahoo y recuperar los

[1] Tomado de https://www.makeuseof.com/stock-price-data-using-python/ (visitado 31/03/2023)

datos asociados con los valores de las acciones de Google en el tiempo, de enero 01 del 2020 a enero 01 del 2022.



Despliegue de procesos en ambientes virtuales a nivel del sistema operativo

Contexto

Como hemos visto, los servicios de red son procesos que corren en un computador pero tienen la capacidad de enviar y recibir mensajes de red. A través de estos mensajes se pueden crear dinámicas de trabajo colaborativo complejas que dan pie al desarrollo de infraestructuras de cómputo escalables y resilientes.

Estos servicios exhiben unas características particulares que les permiten engranarse con otras aplicaciones. Estas características están provistas por librerías ya desarrolladas y probadas largamente y las cuales muchas veces no son fáciles de instalar y/o configurar. Los desarrolladores de servicios de red tienen el *know-how* del despliegue y configuración de estas librerías y logran integrarlas con los servicios de red. Sin embargo, los usuarios finales de estos servicios de red pueden llegar a tener problemas para llevar a cabo este proceso de integración de los servicios con sus librerías (con sus dependencias). Es en este contexto donde aparece la virtualización a nivel del sistema operativo.

Los contenedores son una tecnología de virtualización a nivel del sistema operativo. Docker es uno de los representantes más populares de este tipo de virtualización y permite la integración en un solo archivo de aplicaciones, librerías y dependencias; que permiten el rápido despliegue de servicios de red.

Práctica

Supongamos que queremos distribuir la aplicación que presentamos en la sección anterior. Esta aplicación/script aunque sencilla requiere de una librería externa que permite el consumo de datos disponibles a través de un servicio de Yahoo llamado Yahoo Finance. Adicionalmente, queremos que esta aplicación pueda recibir argumentos a la hora de ser invocada. Aquí tenemos una versión ampliada de dicha aplicación/script

```
# Importing the yfinance package
import yfinance as yf
from datetime import date

#
# Este metodo retorna un pandas dataframe con datos economicos relacionados
# con un 'stock' (e.g. GOOGL, AMZN) entre una fecha inicial (sd) y una fecha
# final (ed). Las fechas pueden estar en formato 'YYYY-MM-DD'.
#
# sd: fecha inicial
# stock: nombre código de la acción
# ed: fecha final. En caso que el usuario no lo indique, se tomara la fecha
# del dia de hoy
#
import sys

def get_stock_data(sd, stock, ed = date.today()):
    return yf.download(stock, sd, ed)

def test(ticker = 'GOOGL'):
    start_date = '2020-01-01'
    return get_stock_data(start_date,ticker)

if __name__ == "__main__":
    ticker = 'GOOGL'
    if len(sys.argv) == 2:
        ticker = sys.argv[1]
    print(f"Se recuperara informacion del stock ${ticker}")
    data = test(ticker)
    print(data.tail())
```

Vamos a crear una imagen de contenedor que nos va a permitir la ejecución de nuestra aplicación desde cualquier entorno computacional que tenga instalado el aplicativo Docker. Para crear la imagen necesitamos de un archivo llamado Dockerfile. Este Dockerfile permite definir aquellos elementos que debe poseer una imagen en Docker.

```
FROM python:3.11.3-bullseye
WORKDIR work
ENV STOCK 'GOOGL'
COPY requirements.txt requirements.txt
RUN pip install -r requirements.txt
COPY . .
CMD ["sh", "-c", "python3 read_value_yf.py ${STOCK}"]
```

Una vez tenemos este 'Dockerfile' procedemos a crear la imagen:

```
docker image build -t stock-market:1.0 .
```

Una vez se tiene la imagen lista se puede llevar a cabo su ejecución

```
docker container run --rm stock-market:1.0
```

Esta sería la salida de correr el contenedor a fecha de 12-05-2023 para el stock 'GOOGL'

```
Se recuperara informacion del stock $GOOGL
[**********************100%**********************]  1 of 1 completed
                 Open        High  ...   Adj Close    Volume
Date                               ...
2023-05-08  105.180000  107.959999  ...  107.769997  26511400
2023-05-09  108.389999  110.150002  ...  107.349998  36360100
2023-05-10  107.970001  112.940002  ...  111.750000  63153400
2023-05-11  115.400002  117.919998  ...  116.570000  78900000
2023-05-12  116.680000  117.779999  ...  117.510002  40443069

[5 rows x 6 columns]
```

Esta sería la salida de correr el contenedor a fecha de 12-05-2023 para el stock 'AMZN[2]

[2] Como pasar argumentos a la ejecución de una imagen de contenedor https://stackoverflow.com/questions/40873165/use-docker-run-command-to-pass-arguments-to-cmd-in-dockerfile

```
docker container run --rm -e STOCK=AMZN stock-market:1.0
```

```
Se recuperara informacion del stock $AMZN
[**********************100%**********************]  1 of 1 completed
                 Open        High  ...   Adj Close    Volume
Date                               ...
2023-05-08  105.040001  106.099998  ...  105.830002  49430900
2023-05-09  105.480003  106.790001  ...  106.620003  44089400
2023-05-10  108.099998  110.669998  ...  110.190002  78627600
2023-05-11  111.029999  113.279999  ...  112.180000  74924800
2023-05-12  112.160004  112.639999  ...  110.260002  49852671

[5 rows x 6 columns]
```

De este modo ya tenemos nuestra aplicación corriendo en un entorno virtual a nivel del sistema operativo.

Creación de servicios a partir de procesos ejecutados en ambientes virtuales

Contexto

- Descripción qué es un proceso y cómo este proceso al permitir ser contactado a través de la red se convierte en un servicio de red.
- Diseñar una gráfica que muestre lo que es un proceso y como un proceso se convierte en un servicio de red.

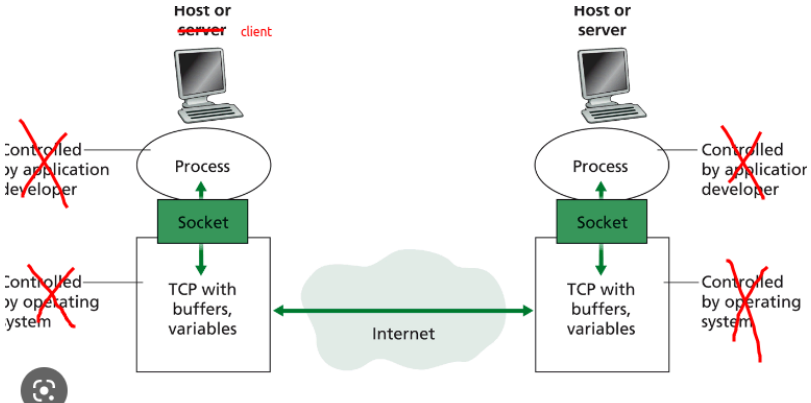


Figure 2.26 ♦ Processes communicating through TCP sockets

Práctica

Se va a modificar el código de la sección Escenario de modo que se pueda acceder a la ejecución del script desde un navegador. Para hacer accesible la funcionalidad de este script desde el navegador se usará el microframework Flask. Usando el código de la aplicación/script de la sección anterior definiremos un nuevo script en Python que invoca el código anterior.

```
from flask import Flask
from datetime import datetime
import read_value_yf

app = Flask(__name__)

@app.route("/")
def hello_world():
    return read_value_yf.test().to_html()

if __name__ == "__main__":
    print(f"{datetime.now()}")
    app.run(host="0.0.0.0")
```

El microframework Flask permite exponer la funcionalidad de un script en Python a través de un API REST. Anteponiendo la instrucción '@app.route(...)' a la definición de una función en Python, esta queda accesible a través del protocolo HTTP.

Ahora se definirá el 'Dockerfile' que nos permitirá crear una imagen que posibilitará el acceso a nuestra aplicación/script desde Python.

```
FROM python:3.11.3-bullseye
WORKDIR work
COPY requirements.txt requirements.txt
RUN pip install -r requirements.txt
COPY . .
EXPOSE 5000
ENV FLASK_ENV=development
CMD ["python3", "app.py"]
```

Se procede a construir la imagen con el comando:

```
docker image build -t stock-market:2.0 .
```

Una vez queda construida la imagen se puede llevar a cabo su ejecución usando el comando:

```
docker container run --rm --name stock-market -p 5000:5000 stock-market:2.0
```

Ahora se puede acceder desde el navegador accediendo al URL http://localhost:5000. Debería tener una salida como la siguiente:

	Open	High	Low	Close	Adj Close	Volume
Date						
2020-01-02	67.420502	68.433998	67.324501	68.433998	68.433998	27278000
2020-01-03	67.400002	68.687500	67.365997	68.075996	68.075996	23408000
2020-01-06	67.581497	69.916000	67.550003	69.890503	69.890503	46768000
2020-01-07	70.023003	70.175003	69.578003	69.755501	69.755501	34330000
2020-01-08	69.740997	70.592499	69.631500	70.251999	70.251999	35314000
2020-01-09	71.096497	71.433998	70.510498	70.989502	70.989502	33200000
2020-01-10	71.473503	71.747002	70.980003	71.447998	71.447998	26258000
2020-01-13	71.762497	72.073997	71.268501	72.001503	72.001503	30730000
2020-01-14	72.000000	72.131500	71.388496	71.529503	71.529503	26076000
2020-01-15	71.651001	72.039001	71.583000	71.959999	71.959999	21550000
2020-01-16	72.272499	72.535004	72.000000	72.508003	72.508003	26080000
2020-01-17	73.126999	74.027496	72.827499	73.975998	73.975998	52424000
2020-01-21	73.949997	74.494003	73.510498	74.112503	74.112503	48930000
2020-01-22	74.486504	75.028999	74.133003	74.193497	74.193497	28458000
2020-01-23	74.377502	74.675003	74.015999	74.234497	74.234497	26650000
2020-01-24	74.640999	74.709999	73.158997	73.308502	73.308502	29786000
2020-01-27	71.577003	71.829002	70.974998	71.586502	71.586502	34132000
2020-01-28	72.086998	72.739998	71.517998	72.525002	72.525002	29724000
2020-01-29	72.922997	73.207001	72.269501	72.834999	72.834999	21566000
2020-01-30	71.904999	72.762497	71.750000	72.712502	72.712502	27110000
2020-01-31	73.392998	73.483498	71.352997	71.639000	71.639000	43822000

2023-05-01	106.839996	107.989998	106.820000	107.199997	107.199997	26681700
2023-05-02	107.139999	107.199997	103.709999	105.320000	105.320000	30997200
2023-05-03	105.529999	107.489998	104.959999	105.410004	105.410004	21795400
2023-05-04	105.489998	105.599998	103.970001	104.690002	104.690002	23419500
2023-05-05	104.820000	105.879997	104.110001	105.570000	105.570000	26625100
2023-05-08	105.180000	107.959999	105.160004	107.769997	107.769997	26511400
2023-05-09	108.389999	110.150002	107.190002	107.349998	107.349998	36360100
2023-05-10	107.970001	112.940002	107.930000	111.750000	111.750000	63153400
2023-05-11	115.400002	117.919998	114.410004	116.570000	116.570000	78900000
2023-05-12	116.680000	118.029999	116.110001	117.510002	117.510002	41073800

Publicación en repositorios públicos de ambientes virtuales

Hasta ahora hemos creado una imagen de contenedor llamada 'stock-market:2.0' que podemos ejecutarla y entonces consultar por el precio de un *stock* o acción en el mercado. Queremos publicar dicha imagen en un repositorio público de imágenes de contenedores de Docker. Para llevar a cabo esta tarea es necesario renombrar la imagen, para ello se ejecuta el comando:

```
docker image tag stock-market:2.0 josanabr/stock-market:2.0
```

josanabr hace referencia al nombre de mi usuario en la plataforma Docker Hub. En su caso, usted deberá cambiar **josanabr** por el nombre de su usuario en Docker Hub.

Una vez renombrada la imagen de contenedor usted podrá subirla. Primero debe conectarse a Docker Hub usando el comando

```
docker login
```

Como ya me había logueado con anterioridad me aparece este mensaje

```
$ docker login
Authenticating with existing credentials...
WARNING! Your password will be stored unencrypted in /home/john/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

Es probable que en su caso le solicite un **Username** y luego un **Password**. Serán entonces su **Username** y **Password** en la plataforma Docker Hub. Una vez autenticado podrá subir la imagen al repositorio de Docker Hub:

```
docker image push josanabr/stock-market:2.0
```

Recuerde que usted deberá cambiar **josanabr** por su login en la plataforma Docker Hub.

Debería aparecer algo como lo siguiente:

```
$ docker image push josanabr/stock-market:2.0
The push refers to repository [docker.io/josanabr/stock-market]
ea5467c2b075: Pushed
2771ccb8bccf: Pushed
e116be917936: Pushed
04f1b86f9f90: Pushed
889828f94722: Mounted from library/python
b0f747a5176b: Mounted from library/python
7dac77253a2b: Mounted from library/python
0007505dc811: Mounted from library/python
f43725f97b9f: Mounted from library/python
9c42af2c6418: Mounted from library/python
d96e248f10e6: Mounted from library/python
d925e0fae4e6: Mounted from library/python
2.0: digest: sha256:dffb56e2d409aee192f45b97ab399d4bbd3d3c7b1464d97047599ab841264114 s
ize: 2841
```

Creación y preparación de entornos virtuales en ambientes de nube pública (máquinas virtuales)

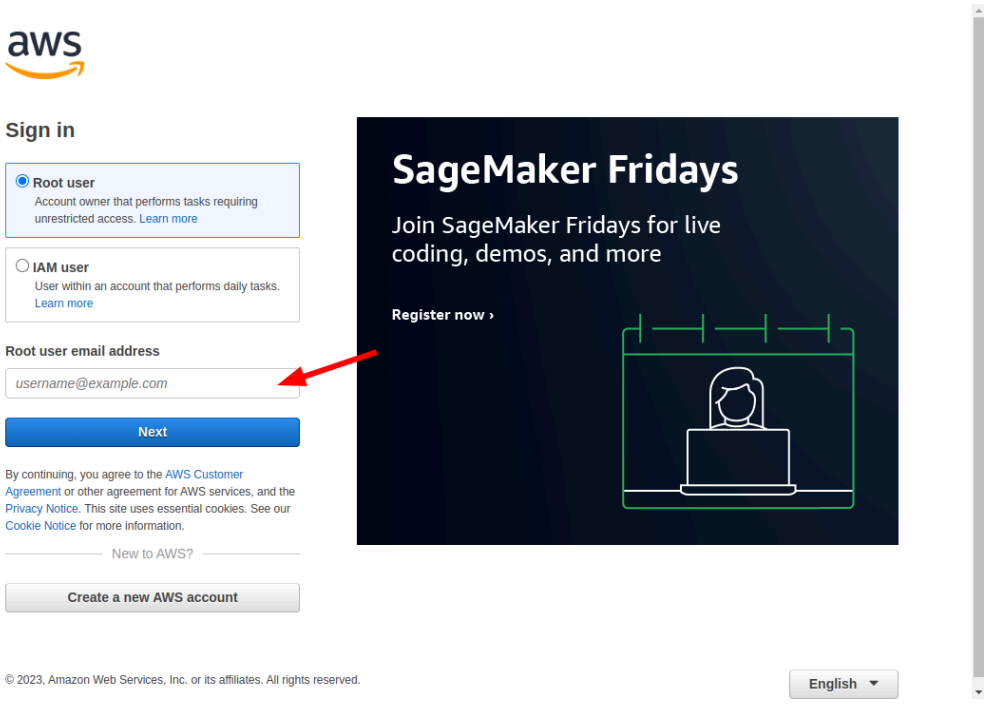
A continuación vamos a preparar un entorno en nube pública que nos va a permitir ejecutar el contendor que hemos creado anteriormente. El entorno de nube que usaremos es el de AWS.

Ingresamos a la consola de AWS (https://aws.amazon.com/console/) y damos click en 'Sign In to the Console'[3]

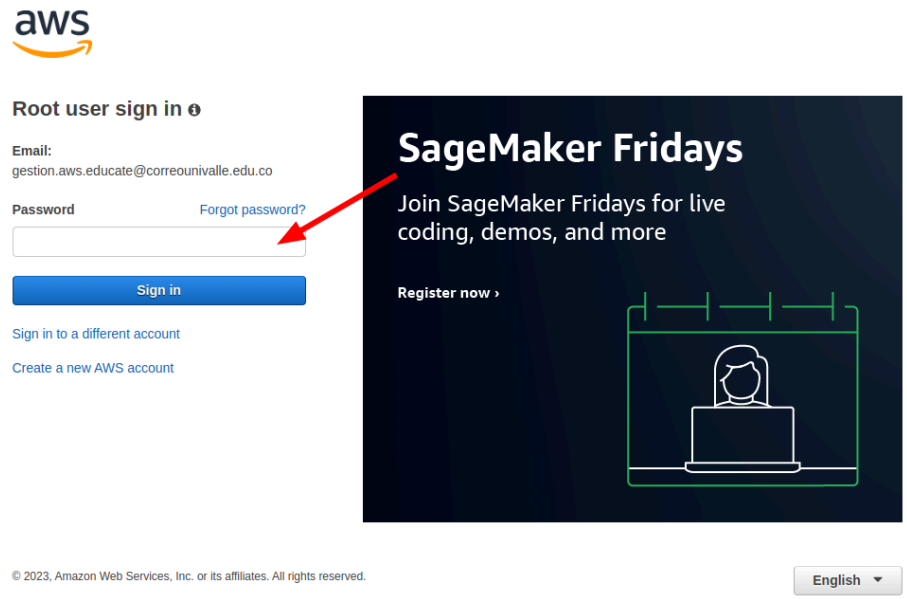


Ingresamos las credenciales de autenticación, *Root user email address*

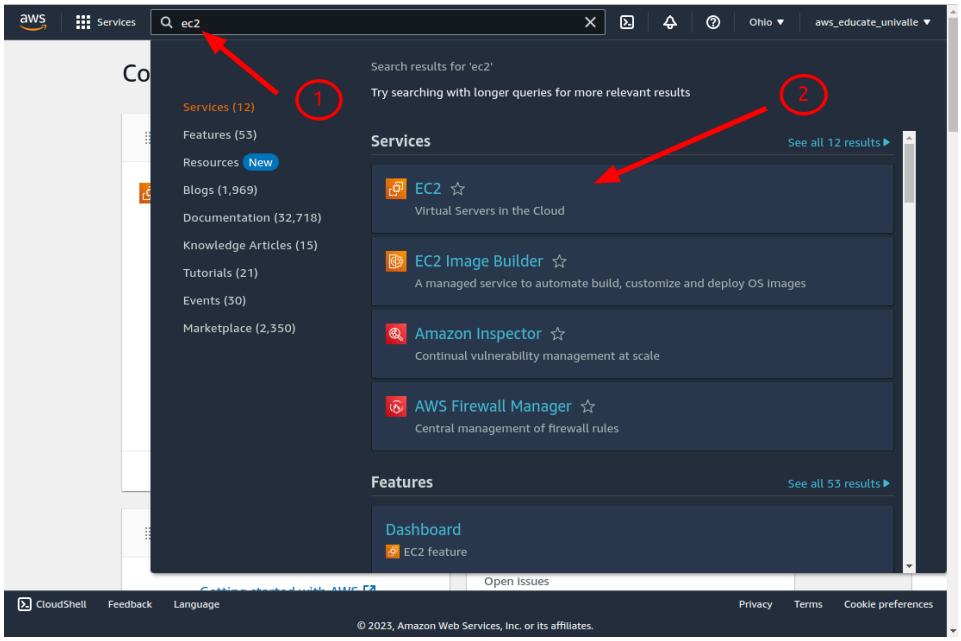
[3] Imágenes tomadas el 25-05-2023


aws
Sign in
Root user
Account owner that performs tasks requiring unrestricted access. Learn more
IAM user
User within an account that performs daily tasks. Learn more
Root user email address
username@example.com
Next
By continuing, you agree to the AWS Customer Agreement or other agreement for AWS services, and the Privacy Notice. This site uses essential cookies. See our Cookie Notice for more information.
New to AWS?
Create a new AWS account
SageMaker Fridays
Join SageMaker Fridays for live coding, demos, and more
Register now ›
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.
English

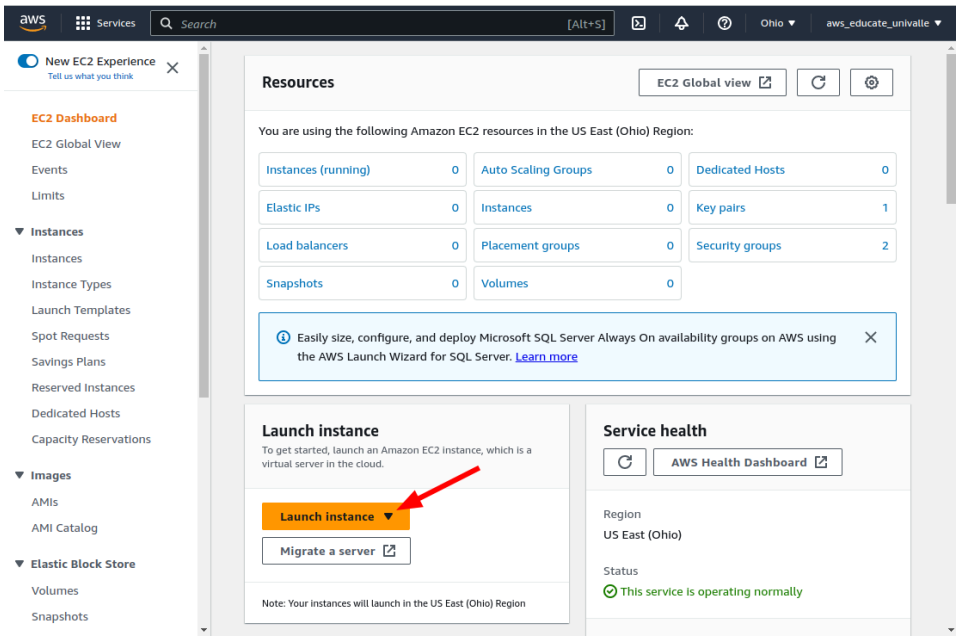
Password


aws
Root user sign in
Email:
gestion.aws.educate@correounivalle.edu.co
Password
Forgot password?
Sign in
Sign in to a different account
Create a new AWS account
SageMaker Fridays
Join SageMaker Fridays for live coding, demos, and more
Register now ›
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.
English

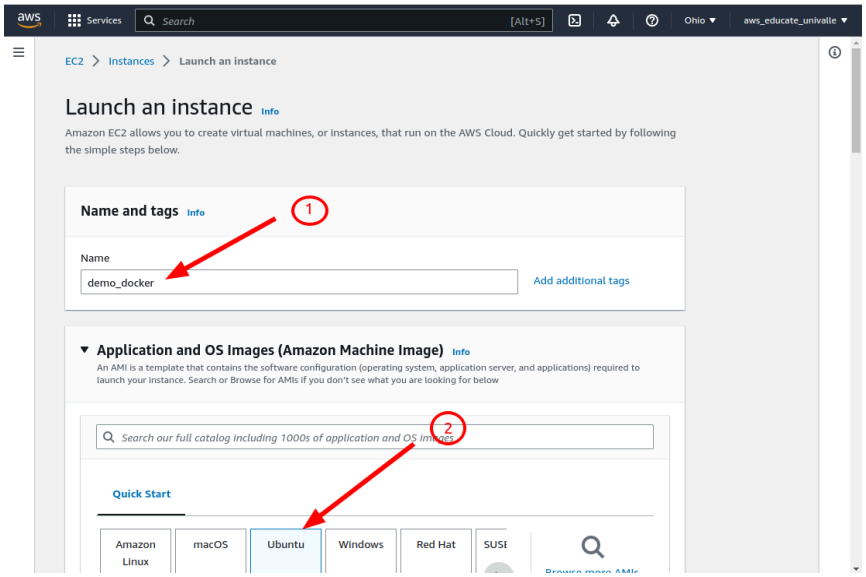
Procedemos a buscar el servicio de AWS llamado EC2 y seleccionamos



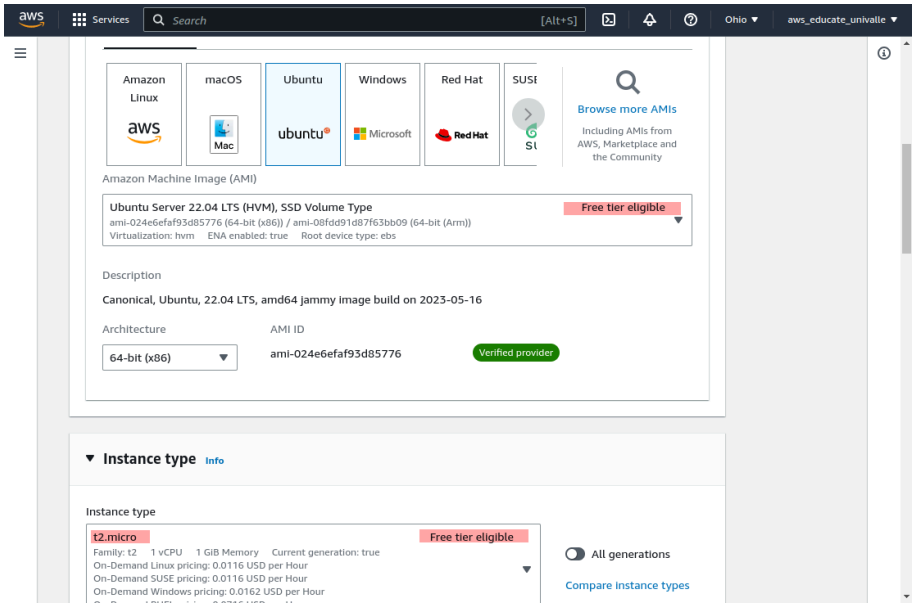
Damos click en '*Launch instance*'



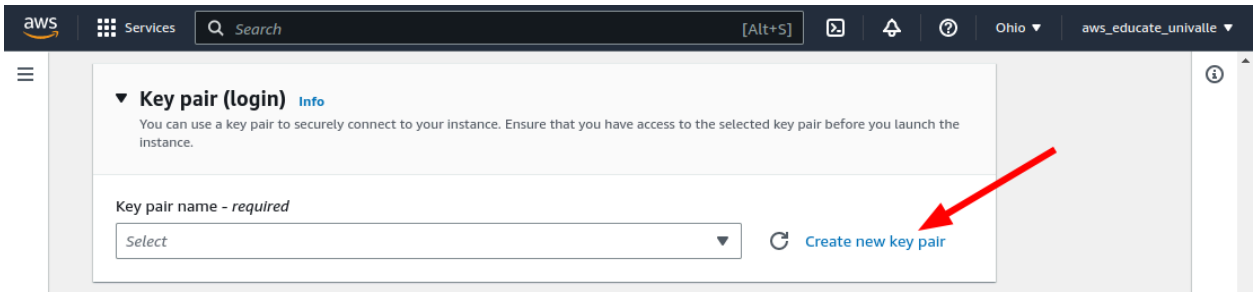
Ahora procedemos a darle un nombre a la instancia (‘demo_docker’) y vamos a seleccionar la imagen de sistema operativo ‘Ubuntu’



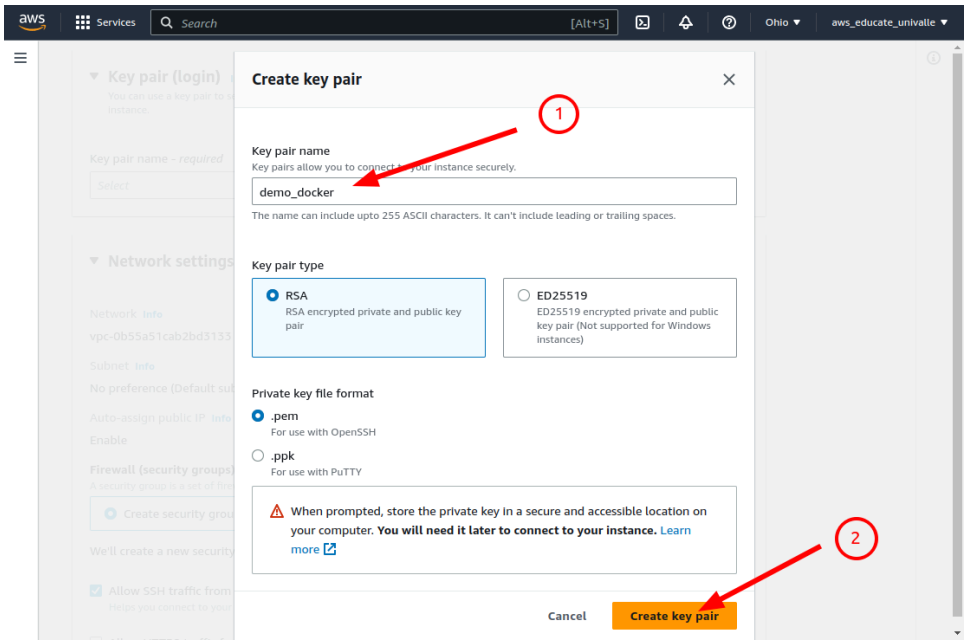
Esta es una imagen la cual es elegible para no ser cobrada por su uso



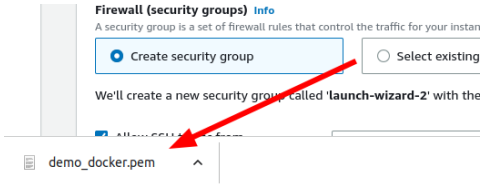
A continuación vamos a crear una llave que nos permita conectarnos a la instancia que vamos a crear en AWS vía el protocolo SSH.



Le damos un nombre a esta nueva llave (‘demo_docker’) y damos posteriormente click en ‘*Create key pair*’

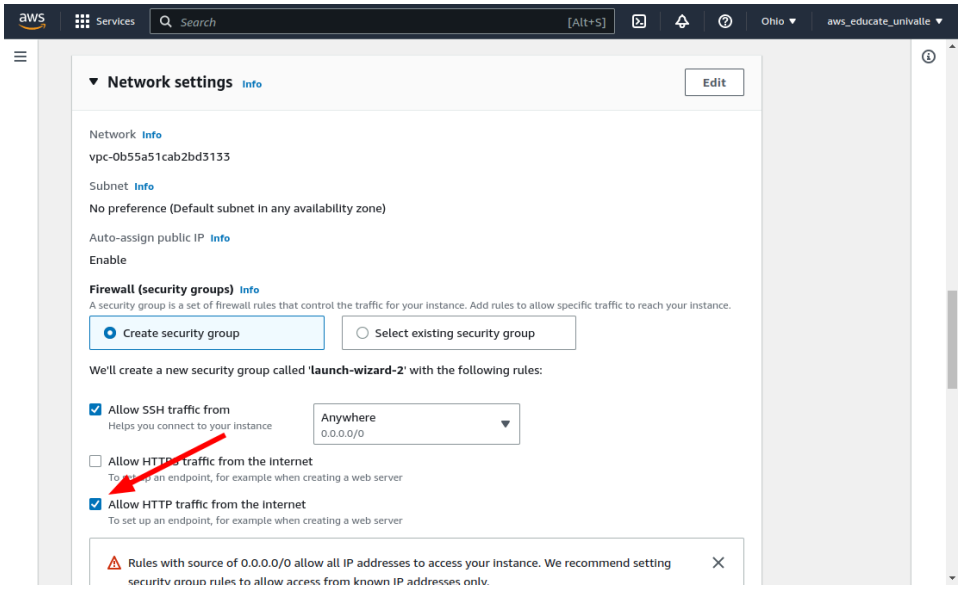


Tu navegador debió descargar un archivo llamado ‘demo_docker.pem’

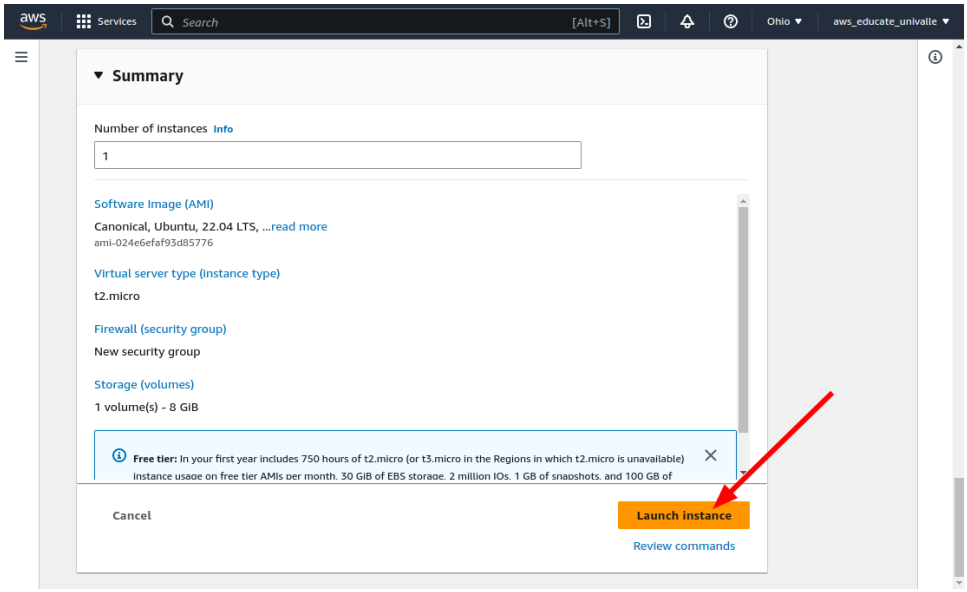


Este archivo se usará más tarde para llevar a cabo la ejecución de nuestro contenedor en la instancia que vamos a ejecutar en un momento.

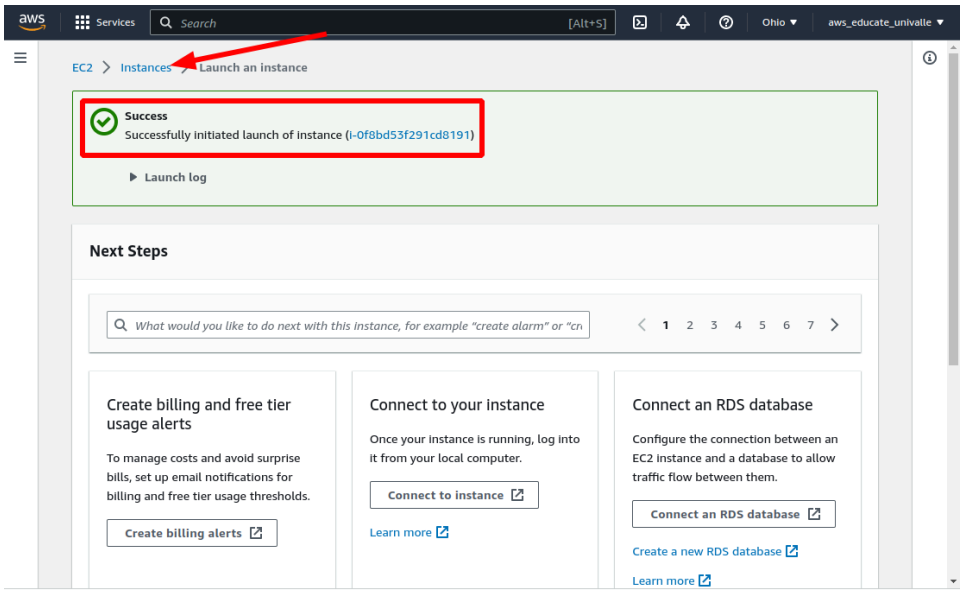
En la sección de ‘*Network settings*’ se habilita la opción ‘*Allow HTTP traffic from the internet*’ ya que nuestro servicio estará disponible a través del protocolo HTTP.



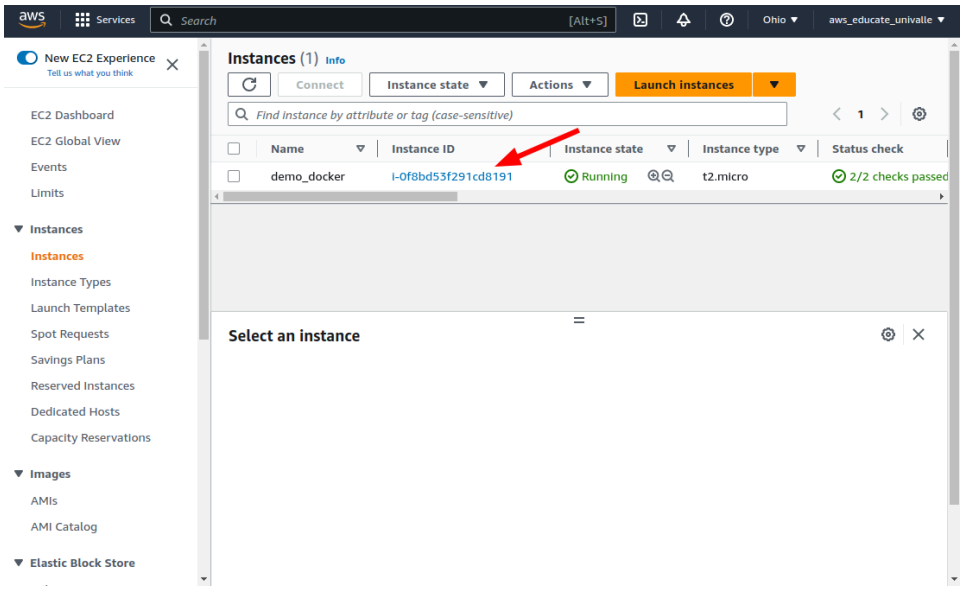
Ya estamos listos para crear nuestra máquina virtual en AWS.



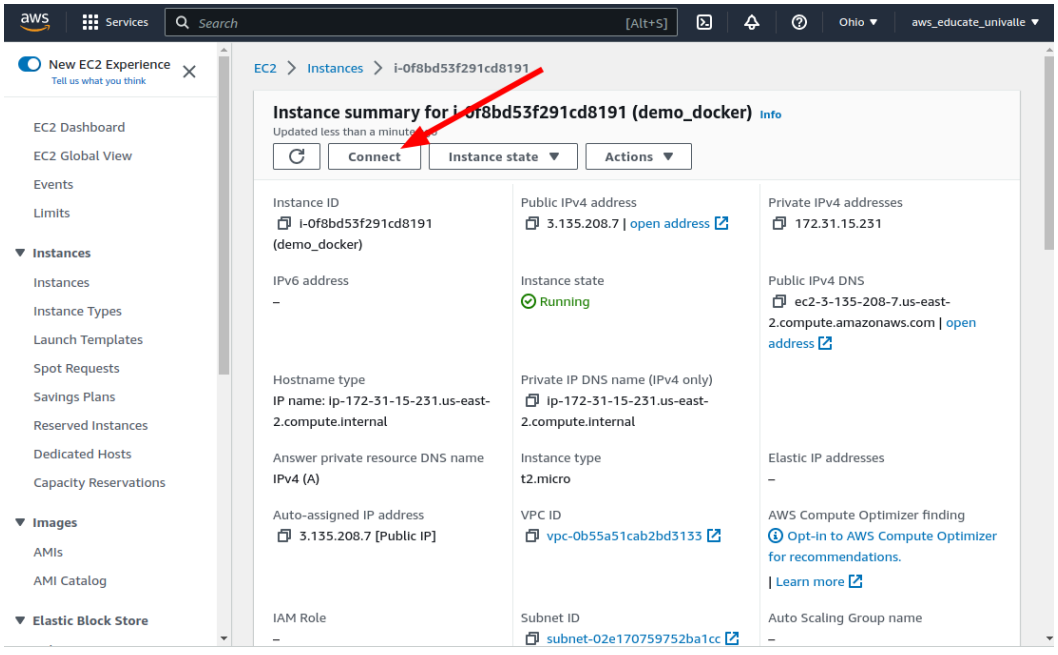
Nuestra instancia se creó exitosamente y vamos a la sección de '*Instances*'



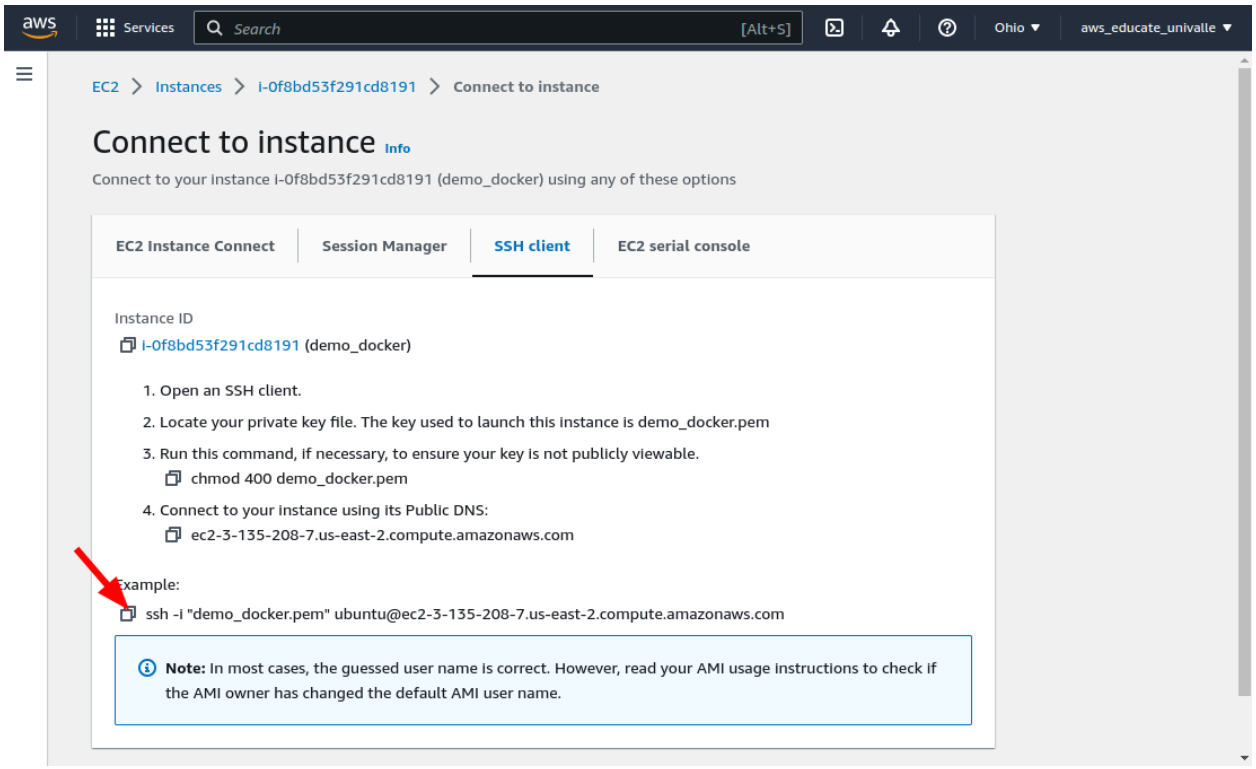
Vemos que nuestra imagen está en ejecución y damos click sobre el '*Instance ID*' de esta máquina.



Damos click ahora en '*Connect*'



Se nos indican una serie de tareas que se deben ejecutar para llevar a feliz término la conexión a través de SSH.



Los pasos de como conectarse a la instancia en [este enlace](#).

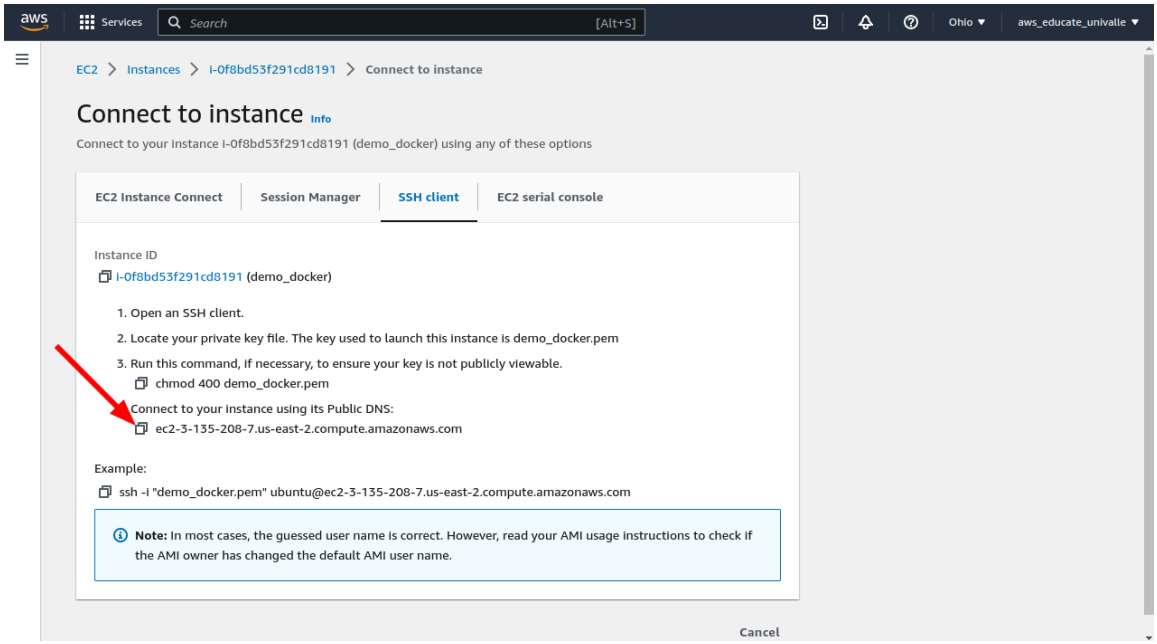
Acondicionamiento de la máquina virtual

Para acondicionar la máquina virtual se debe:

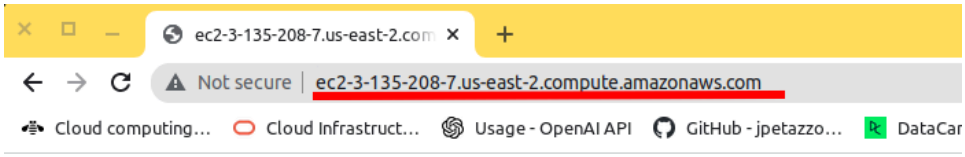
- Instalar Docker, (tomado de este sitio → https://docs.docker.com/engine/install/ubuntu/)
- Habilitar al usuario 'ubuntu' para ejecutar contenedores
- Descargar el contenedor que creamos anteriormente y ejecutarlo

En este video se muestra como llevar a cabo estos pasos.

Ahora, para validar que nuestro servicio esté operando copiamos el nombre de nuestro servidor



Y abrimos una ventana de navegador hacia ese sitio



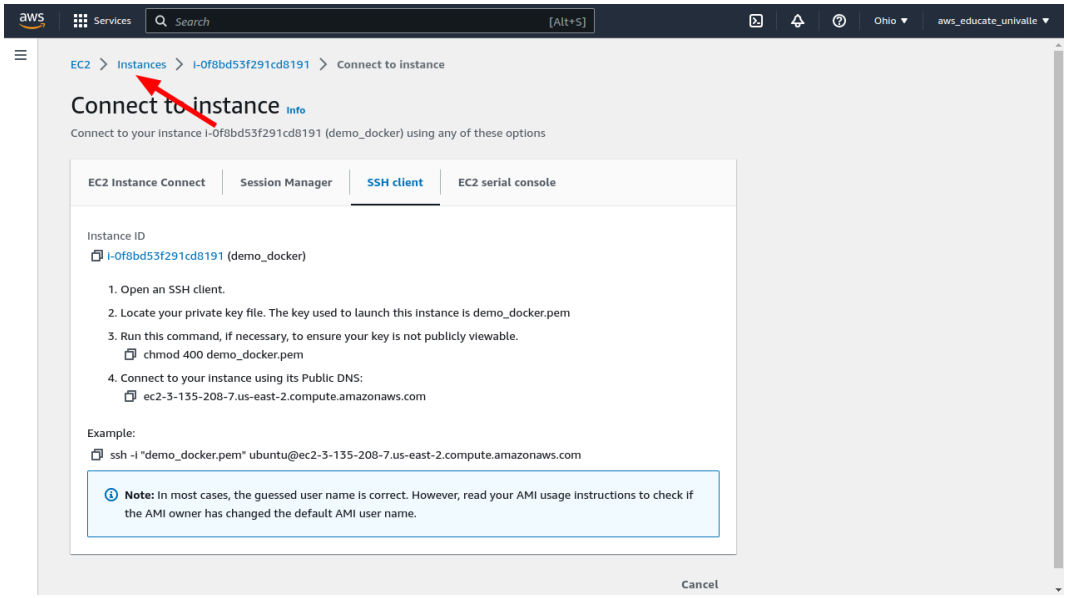
	Open	High	Low	Close	Adj Close	Volume
Date						
2020-01-02	67.420502	68.433998	67.324501	68.433998	68.433998	27278000
2020-01-03	67.400002	68.687500	67.365997	68.075996	68.075996	23408000
2020-01-06	67.581497	69.916000	67.550003	69.890503	69.890503	46768000
2020-01-07	70.023003	70.175003	69.578003	69.755501	69.755501	34330000
2020-01-08	69.740997	70.592499	69.631500	70.251999	70.251999	35314000
2020-01-09	71.096497	71.433998	70.510498	70.989502	70.989502	33200000
2020-01-10	71.473503	71.747002	70.980003	71.447998	71.447998	26258000
2020-01-13	71.762497	72.073997	71.268501	72.001503	72.001503	30730000
2020-01-14	72.000000	72.131500	71.388496	71.529503	71.529503	26076000
2020-01-15	71.651001	72.039001	71.583000	71.959999	71.959999	21550000
2020-01-16	72.272499	72.535004	72.000000	72.508003	72.508003	26080000
2020-01-17	73.126999	74.027496	72.827499	73.975998	73.975998	52424000
2020-01-21	73.949997	74.494003	73.510498	74.112503	74.112503	48930000
2020-01-22	74.486504	75.028999	74.133003	74.193497	74.193497	28458000
2020-01-23	74.377502	74.675003	74.015999	74.234497	74.234497	26650000
2020-01-24	74.640999	74.709999	73.158997	73.308502	73.308502	29786000
2020-01-27	71.577003	71.829002	70.974998	71.586502	71.586502	34132000
2020-01-28	72.086998	72.739998	71.517998	72.525002	72.525002	29724000
2020-01-29	72.922997	73.207001	72.269501	72.834999	72.834999	21566000
2020-01-30	71.904999	72.762497	71.750000	72.712502	72.712502	27110000
2020-01-31	73.392998	73.483498	71.352997	71.639000	71.639000	43822000
2020-02-03	73.082497	74.315002	72.830498	74.129997	74.129997	71636000
2020-02-04	72.724503	73.366997	71.101501	72.270500	72.270500	95880000
2020-02-05	73.180496	73.228996	71.484001	72.302498	72.302498	36376000
2020-02-06	72.598999	74.078003	72.524002	73.798500	73.798500	37822000
2020-02-07	73.369499	74.262497	73.270500	73.955498	73.955498	28360000
2020-02-10	73.861504	75.444000	73.760002	75.432999	75.432999	38302000
2020-02-11	75.663498	76.494003	75.272499	75.502998	75.502998	34852000
2020-02-12	75.792999	76.014999	75.433502	75.931503	75.931503	31042000

Ya tenemos nuestro servicio corriendo en una nube pública.

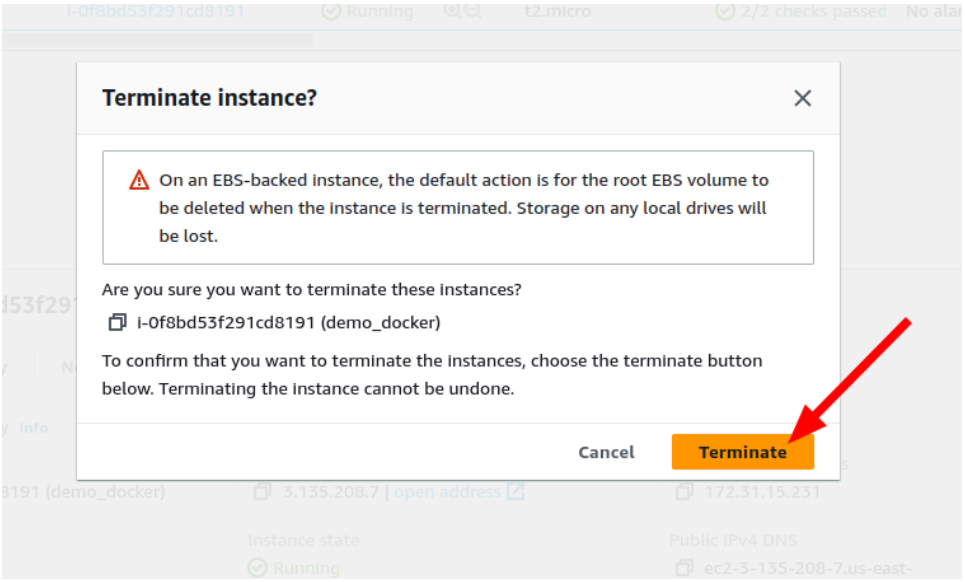
Finalmente y no menos importante, de hecho MUY IMPORTANTE

Apaguemos la máquina virtual para que no se nos cobre dinero extra por uso de los recursos en AWS

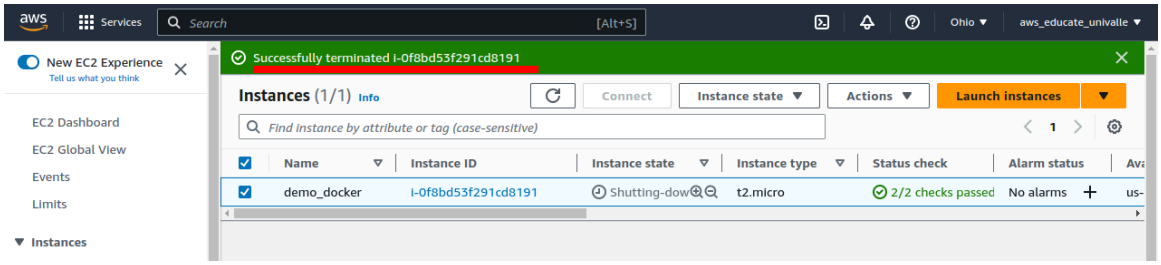
Vamos a '*Instances*'



Seleccionamos nuestra instancia, damos luego click en '*Instance state*' y finalmente click en '*Terminate instance*'. Confirmamos que la deseamos terminar



Y deberíamos tener un mensaje como este



No se preocupe si la instancia no aparece de la lista de instancias de su interfaz web. En cuestión de unas horas, AWS la borrará. Por ahora, es seguro que esta instancia no consumirá créditos.

Referencias

- https://docker-curriculum.com/
- https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/GettingStarted.CreateApp.html
- https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/create_deploy_docker_v2config.html → Dockerrun.aws.json